

Linux OS Management Script Suite Project Report

Nada Elshami & Riaa Sehgal

SYST44288

Operating Systems Design & Systems Programming

Gurdeep Gill



Part A — `process_manager_tool.sh`

This shell script is designed to help users monitor and manage system processes. It can list running processes, filter them by user, kill processes by PID, display top CPU/memory consumers, and log process data every minute.

1. List all running processes

```
===== PROCESS MANAGER TOOL =====
1) List all running processes
2) Kill a process by PID
3) Display processes by specific user
4) Show top 5 CPU and Memory consuming processes
5) Start logging process status every 1 minute
6) Exit

Enter your choice: 1
=====
PID USER      %CPU  %MEM  COMMAND
PID USER      %CPU  %MEM  COMMAND
1721 nada      4.3    6.6    /usr/bin/gnome-shell
2972 nada      2.1    1.3    /usr/libexec/gnome-terminal-server
3610 nada      1.6    5.9    /usr/bin/python3
1863 nada      0.8    0.3    /usr/bin/ibus-daemon
1306 root      0.5    1.3    /usr/libexec/packagekitd
2058 nada      0.5    0.3    /snap/snap-store/582/usr/bin/snap-store
2041 nada      0.3    0.2    /usr/libexec/ibus-engine-simple
867  root      0.2    1.0    /usr/lib/snapd/snapd
1500 nada      0.2    0.6    /usr/bin/pulseaudio
4040 root      0.2    0.6    [kworker/u258:0-events_unbound]
1  root      0.1    0.3    /sbin/init
25  root      0.1    0.0    [kworker/i:0-events]
651 systemd+ 0.1    0.1    /lib/systemd/systemd-oomd
737  root      0.1    0.2    /usr/bin/vmtoolsd
Terminal nada      0.1    0.6    /usr/libexec/ibus-extension-gtk3
1  nada      0.1    0.9    /usr/bin/vmtoolsd
2  root      0.0    0.0    [kthread]
3  root      0.0    0.0    [pool_workqueue_release]
4  root      0.0    0.0    [kworker/R-rcu_g]
5  root      0.0    0.0    [kworker/R-rcu_p]
6  root      0.0    0.0    [kworker/R-slub_]
7  root      0.0    0.0    [kworker/R-netns]
8  root      0.0    0.0    [kworker/0:0-cgwb_release]
12 root      0.0    0.0    [kworker/R-mm_pg]
13 root      0.0    0.0    [rcu_tasks_kthread]
14 root      0.0    0.0    [rcu_tasks_rude_kthread]
15 root      0.0    0.0    [rcu_tasks_trace_kthread]
16 root      0.0    0.0    [ksoftrigd/0]
17 root      0.0    0.0    [rcu_preempt]
18 root      0.0    0.0    [migration/0]
19 root      0.0    0.0    [idle_inject/0]
```

2. Show top 5 CPU & memory-heavy processes

```
nada@nada-virtual-machine: ~/OS_Project

===== PROCESS MANAGER TOOL =====
1) List all running processes
2) Kill a process by PID
3) Display processes by specific user
4) Show top 5 CPU and Memory consuming processes
5) Start logging process status every 1 minute
6) Exit

Enter your choice: 4
Top 5 CPU consuming processes:
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
nada      1721  4.0   6.6 4433468 262256 ?        Ssl  17:32   1:20 /usr/bin/gnome-shell
nada      2972  1.9   1.3 557108 53892 ?        Ssl  17:33   0:38 /usr/libexec/gnome-terminal-server
nada      3610  1.4   5.9 721008 236420 ?        Ssl  17:33   0:26 /usr/bin/python3 /usr/bin/update-manager --no-update --no-focus-on-map
nada      1863  0.7   0.3 389296 13248 ?        Sl   17:32   0:15 /usr/bin/ibus-daemon --panel disable
root      1306  0.5   1.3 372860 52732 ?        Ssl  17:29   0:11 /usr/libexec/packagekitd

Top 5 Memory consuming processes:
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
nada      1721  4.0   6.6 4433468 262256 ?        Ssl  17:32   1:20 /usr/bin/gnome-shell
nada      2058  0.4   6.3 1402152 253140 ?        Sl   17:32   0:09 /snap/snap-store/582/usr/bin/snap-store --gapplication-service
nada      3610  1.4   5.9 721008 236420 ?        Ssl  17:33   0:26 /usr/bin/python3 /usr/bin/update-manager --no-update --no-focus-on-map
nada      2148  0.0   2.0 235420 79556 ?        S    17:32   0:00 /usr/bin/Xwayland :0 -rootless -noreset -accessx -core -auth /run/user/1000/.nutter-Xwaylandauth.SQJ6E3 -listen 4
nada      1907  0.0   1.6 838196 63736 ?        Sl   17:32   0:00 /usr/libexec/evolution-data-server/evolution-alarm-notify

Press ENTER to continue...
```

3. Show processes for your username (nada)

```
nada@nada-virtual-machine: ~/OS_Project

===== PROCESS MANAGER TOOL =====
1) List all running processes
2) Kill a process by PID
3) Display processes by specific user
4) Show top 5 CPU and Memory consuming processes
5) Start logging process status every 1 minute
6) Exit
=====
Enter your choice: 3
Enter username: nada
Processes for nada:
  PID USER      %CPU %MEM    COMMAND
  1551 nada        0.0  0.2  /lib/systemd/systemd --user
  1552 nada        0.0  0.1  (sd-pam)
  1558 nada        0.0  0.1  /usr/bin/pipewire
  1559 nada        0.0  0.1  /usr/bin/pipewire-media-session
  1560 nada        0.2  0.6  /usr/bin/pulseaudio --daemonize=no --log-target=journal
  1572 nada        0.0  0.1  /usr/bin/gnome-keyring-daemon --daemonize --login
  1584 nada        0.0  0.1  /usr/bin/dbus-daemon --session --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
  1585 nada        0.0  0.1  /usr/libexec/xdg-document-portal
  1589 nada        0.0  0.1  /usr/libexec/xdg-permission-store
  1616 nada        0.0  0.1  /usr/libexec/gdm-wayland-session env GNOME_SHELL_SESSION_MODE=ubuntu /usr/bin/gnome-session --session=ubuntu
  1619 nada        0.0  0.3  /usr/libexec/gnome-session-binary --session=ubuntu
  1671 nada        0.0  0.1  /usr/libexec/gnome-session-ctl --monitor
  1683 nada        0.0  0.1  /usr/libexec/gvfsd
  1689 nada        0.0  0.1  /usr/libexec/gvfsd-fuse /run/user/1000/gvfs -f
  1690 nada        0.0  0.4  /usr/libexec/gnome-session-binary --systemd-service --session=ubuntu
  1718 nada        0.0  0.1  /usr/libexec/at-spi-bus-launcher --launch-immediately
  1721 nada        4.1  6.6  /usr/bin/gnome-shell
  1730 nada        0.0  0.1  /usr/bin/dbus-daemon --config-file=/usr/share/defaults/at-spi2/accessibility.conf --nofork --print-address 11 --address=unix:path=/run/user/1000/at-spi/b
  1749 nada        0.0  0.5  /usr/libexec/gnome-shell-calendar-server
  1755 nada        0.0  0.6  /usr/libexec/evolution-source-registry
  1761 nada        0.0  0.1  /usr/libexec/dconf-service
  1764 nada        0.0  0.9  /usr/libexec/goa-daemon
  1770 nada        0.0  0.2  /usr/libexec/gvfs-udisks2-volume-monitor
  1778 nada        0.0  0.7  /usr/libexec/evolution-calendar-factory
  1779 nada        0.0  0.1  /usr/libexec/gvfs-ntp-volume-monitor
  1785 nada        0.0  0.2  /usr/libexec/gvfs-afc-volume-monitor
  1790 nada        0.0  0.1  /usr/libexec/gvfs-goa-volume-monitor
  1801 nada        0.0  0.7  /usr/libexec/evolution-addressbook-factory
  1810 nada        0.0  0.3  /usr/libexec/goa-identity-service
  1815 nada        0.0  0.1  /usr/libexec/gvfs-gphoto2-volume-monitor
```

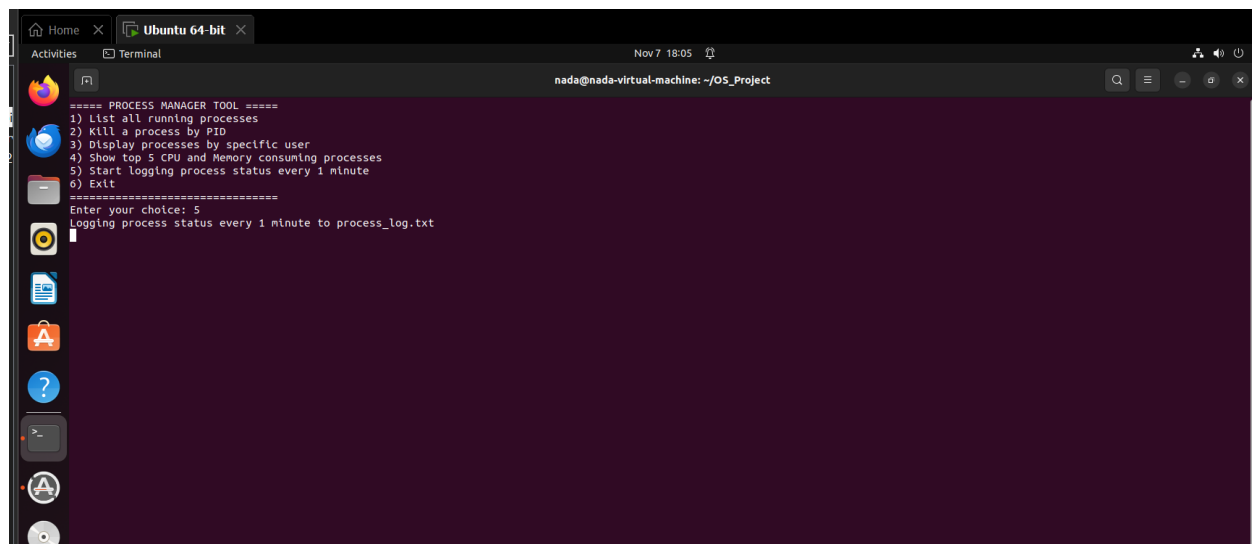
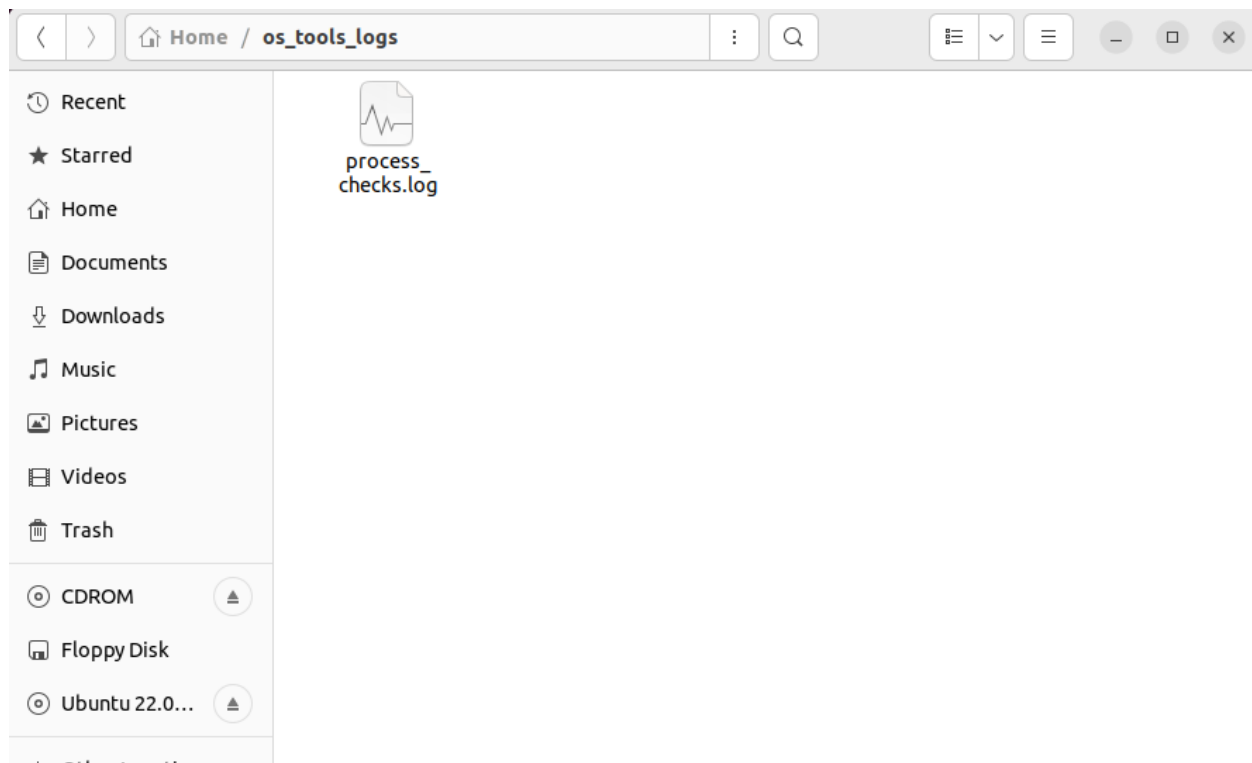
4. Kill a process (example PID = 2294)

```
Nov 7 18:04

nada@nada-virtual-machine: ~/OS_Project

===== PROCESS MANAGER TOOL =====
1) List all running processes
2) Kill a process by PID
3) Display processes by specific user
4) Show top 5 CPU and Memory consuming processes
5) Start logging process status every 1 minute
6) Exit
=====
Enter your choice: 2
Enter PID to kill: 2288
Process 2288 terminated successfully.
Press ENTER to continue..
```

5. The Logging Feature (the scheduling part)



Report for Part A

1. *ps aux*: Lists all active processes with details like PID, user, CPU, memory, and command. Used to show all processes and sort them by CPU or memory usage.
2. Sorting with *ps aux --sort*: Used to display top 5 resource-consuming processes.
 - a. *--sort=-%cpu* → this sorts the highest CPU usage first
 - b. *--sort=-%mem* → this sorts the highest memory usage first

3. *awk*: Formats output for readability, allowing aligned columns for PID, user, CPU, memory, and command.
4. *kill*: Terminates a process using its PID. Used when the user selects the “Kill a process” option.
5. *ps -u*: Shows only processes belonging to a specific user. Used for filtering processes by username.
6. *head -n 6*: This would show only the top 5 processes with their headers. It is used after sorting CPU and memory usage.
7. *>> process_log.txt*: Appends full process snapshots to a log file during automated monitoring.
8. *sleep 60*: It creates a 1-minute wait period between log entries.
9. *while true*: Runs continuous logging until manually stopped.

Learning Outcomes

This project helped us gain a better understanding of how Linux manages system processes, including how CPU or memory consumption, user ownership, and PIDs affect overall system performance. My confidence in navigating and managing processes directly from the terminal has increased because of working with commands like *ps*, *awk*, and *kill*. I also had practical experience monitoring system resources, identifying processes that use too much CPU or memory, and automating routine process logging by writing the script. Additionally, creating a menu-driven tool helped me become more proficient with Bash scripting, particularly about arranging user input and clearly presenting information. All things considered, this assignment offered insightful, useful information about the daily tasks associated with system administration and cybersecurity work.

Part B — *cpu_manager_tool.sh*

This shell script was created to help monitor, analyze, and manage CPU usage on a system. It provides key functionality such as displaying current CPU usage, logging usage trends over time, assigning CPU affinity to specific processes, and generating alerts when CPU usage exceeds a defined threshold.

1. Display the current CPU usage percentage.

```

===== CPU Manager Tool =====
1) Display current CPU usage
2) Log CPU usage over time
3) Set CPU affinity for a process
4) Check & alert if CPU usage exceeds threshold (90%)
5) Exit
Choose an option: 1
Current CPU Usage: 1%

```

2. Track CPU usage over a specific period and log the data

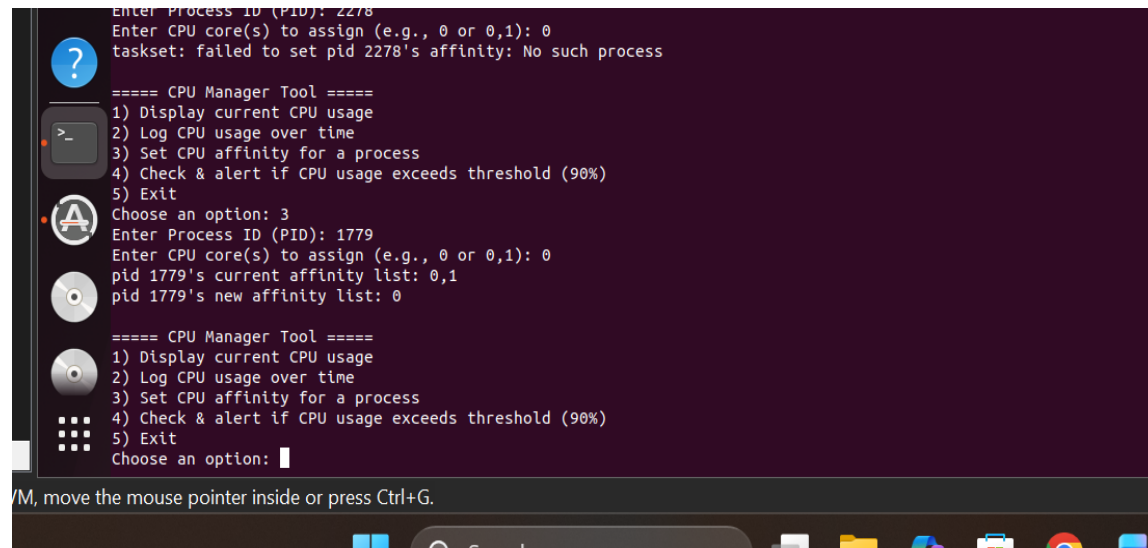
```

===== CPU Manager Tool =====
1) Display current CPU usage
2) Log CPU usage over time
3) Set CPU affinity for a process
4) Check & alert if CPU usage exceeds threshold (90%)
5) Exit
Choose an option: 2
How many times should I log?
3
Logging CPU usage to cpu_usage.log...
Logged: 1%
Logged: 3%
Logged: 0%
Done. Log saved to cpu_usage.log

===== CPU Manager Tool =====
1) Display current CPU usage
2) Log CPU usage over time
3) Set CPU affinity for a process
4) Check & alert if CPU usage exceeds threshold (90%)
5) Exit
Choose an option:

```

3. Allow users to set CPU affinity for a specific process (assign it to specific CPUs)



4. Generate an alert if CPU usage exceeds a certain threshold (e.g., 90%).

```
==== CPU Manager Tool ====
1) Display current CPU usage
2) Log CPU usage over time
3) Set CPU affinity for a process
4) Check & alert if CPU usage exceeds threshold (90%)
5) Exit
Choose an option: 4
Current Usage: 1%
CPU usage is stable.
```

Report for Part B

1. `grep 'cpu ' /proc/stat`: It reads raw CPU statistics directly from the kernel. Used to calculate CPU usage by comparing idle and total CPU times over a one-second interval.
2. `taskset`: Controls CPU affinity, allowing a process to run only on specific CPU cores. Used to let the user assign a process (PID) to specific CPU cores such as 0, 1, or multiple cores like 0,1.
3. `>> cpu_usage.log`: It appends timestamped CPU usage data to a log file. Used to track CPU performance over a user-defined number of intervals.
4. `sleep 1`: This would create a delay between readings when calculating CPU usage and logging. It ensures accurate CPU calculation and properly spaced log entries.

Learning Outcomes

Through this project, we strengthened our understanding of how Linux measures system performance at the CPU level. We learned how the kernel monitors CPU statuses like idle, user, and system time by reading data straight from `/proc/stat`. Additionally, we learnt how CPU affinity functions and how administrators can regulate the number of cores that a process is permitted to use, which is a useful tool for performance optimization. We were able to comprehend the importance of monitoring CPU utilization trends, particularly when identifying system slowdowns, thanks to the implementation of real-time logging.

Part C — *memory_manager_tool.sh*

This shell script was created to monitor, analyze, and manage system memory in a Linux environment. It provides essential features such as displaying current memory usage, listing processes that exceed a specified memory threshold, clearing cached memory, and generating alerts when available memory drops below a user-defined limit. It helps users better understand memory behavior and respond to low-memory conditions effectively.

1. Displaying current memory usage

```
nada@nada-virtual-machine:~$ cd ~/scripts
pwd
bash: cd: /home/nada/scripts: No such file or directory
/home/nada
nada@nada-virtual-machine:~$ ls
buffer.txt  Downloads  mypipe    OS_Project  Pictures  Videos
deadlock_lab  hello     Orphan    OS_Project.zip  Public    zombie
Desktop     hello.c   orphan    os_tools_logs  snap      Zombie.c
Documents    Music     Orphan.c  os_tools_logs.zip  Templates zombie.c
nada@nada-virtual-machine:~$ cd ~/OS_Project
nada@nada-virtual-machine:~/OS_Project$ ls
cpu_manager_tool.sh  cpu_usage.log  process_log.txt  process_manager_tool.sh
nada@nada-virtual-machine:~/OS_Project$ pwd
/home/nada/OS_Project
nada@nada-virtual-machine:~/OS_Project$ nano memory_manager_tool.sh
nada@nada-virtual-machine:~/OS_Project$ chmod +x memory_manager_tool.sh
nada@nada-virtual-machine:~/OS_Project$ ./memory_manager_tool.sh

===== Memory Manager Tool =====
1) Display current memory usage
2) List processes above a memory threshold
3) Clear cache and buffers (requires root)
4) Alert on low available memory
0) Exit
Choose an option: 1
=== Current memory usage (free -m) ===
```

	total	used	free	shared	buff/cache	available
Mem:	3868	1144	679	39	2043	2424
Swap:	2139	0	2139			

2. Processes above memory threshold

```
===== Memory Manager Tool =====
1) Display current memory usage
2) List processes above a memory threshold
3) Clear cache and buffers (requires root)
4) Alert on low available memory
0) Exit
Choose an option: 2
Show processes using more than how many MB? 100
=== Processes using more than 100MB of RAM ===
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
nada	1694	1.4	6.8	4427728	272920	?	Ssl	13:36	0:39	/usr/bin/gnome-shell
nada	2068	0.2	4.6	1322168	184116	?	Sl	13:36	0:08	/snap/snap-store/582/usr/bin/snap-store --gapplication-service

```
===== Memory Manager Tool =====
```

3. Clear cache

```
pyc:
nada@nada-virtual-machine:~/OS_Project$ sudo ./memory_manager_tool.sh
[sudo] password for nada:

===== Memory Manager Tool =====
1) Display current memory usage
2) List processes above a memory threshold
3) Clear cache and buffers (requires root)
4) Alert on low available memory
0) Exit
Choose an option: 3
This action clears filesystem caches and needs root privileges.
Syncing and dropping caches...
Caches cleared.
```

4. Set an alert for low available memory (e.g., below 100MB).


```

===== Memory Manager Tool =====
1) Display current memory usage
2) List processes above a memory threshold
3) Clear cache and buffers (requires root)
4) Alert on low available memory
0) Exit
Choose an option: 4
Low memory threshold in MB (default 100):
Alert will trigger if available memory < 100MB.
Press Ctrl + C to stop.
[Wed 26 Nov 2025 02:23:18 PM EST] Available memory: 2423MB
[Wed 26 Nov 2025 02:23:23 PM EST] Available memory: 2423MB
[Wed 26 Nov 2025 02:23:28 PM EST] Available memory: 2423MB
[Wed 26 Nov 2025 02:23:33 PM EST] Available memory: 2423MB

```

Report for Part C

1. *ps aux* with *awk*: Lists all running processes, and the script uses *awk* to convert RSS (resident memory) from KB to MB. This helps identify processes consuming more than a specified amount of RAM.
2. *grep + /proc/meminfo*: Reads memory information directly from the kernel. The script uses the *MemAvailable* value to check how much memory is truly available at any moment.
3. *sync and echo 3 > /proc/sys/vm/drop_caches*: Clears filesystem caches, dentries, and inodes. This requires root privileges and is used to free reclaimable memory without rebooting the system.
4. *>> memory_alerts.log*: Appends timestamped low-memory alerts to a log file, allowing tracking of when memory shortages occur.
5. *sleep 5*: Provides a delay between each memory check during the alert loop, preventing excessive system calls and allowing readable monitoring.

Learning Outcomes

Through this project, we strengthened our understanding of how Linux manages and reports system memory. We learned how tools like *free*, */proc/meminfo*, and process statistics help administrators gauge memory pressure and detect resource-heavy applications. Implementing threshold-based monitoring improved our awareness of how low-memory situations can affect system performance. Additionally, we gained practical insight into cache clearing and why administrators might use it during troubleshooting or performance optimization.

Part D — *file_system_monitor_tool.sh*

This shell script was created to help monitor disk usage, analyze file sizes, and maintain filesystem health. It includes essential features such as displaying disk usage for all mounted filesystems, finding the largest files inside a chosen directory, listing files modified in the last 24 hours, and cleaning temporary files when a directory exceeds a specified size limit. This tool supports both general system monitoring and cleanup tasks that help keep storage usage under control.

1. Display disk usage for each mounted filesystem.

```
===== File System Monitor Tool =====
1) Display disk usage for each mounted filesystem
2) List the top 15 largest files in a directory
3) Show files modified in the last 24 hours
4) Clean temporary files if directory size exceeds a limit
0) Exit
Choose an option: 1
=== Disk usage (df -h) ===
Filesystem      Size  Used Avail Use% Mounted on
tmpfs            387M  2.0M  385M   1% /run
/dev/sda3        20G   13G   5.8G  69% /
tmpfs            1.9G   0    1.9G   0% /dev/shm
tmpfs            5.0M  4.0K   5.0M   1% /run/lock
/dev/sda2        512M   6.1M  506M   2% /boot/efi
tmpfs            387M  2.4M  385M   1% /run/user/1000
/dev/sr0         127M  127M    0 100% /media/nada/CDROM
/dev/fd0         1.4M   9.0K   1.4M   1% /media/floppy0
```

2. List the top 15 largest files in a specified directory

```
===== File System Monitor Tool =====
1) Display disk usage for each mounted filesystem
2) List the top 15 largest files in a directory
3) Show files modified in the last 24 hours
4) Clean temporary files if directory size exceeds a limit
0) Exit
Choose an option: 2
Enter directory to scan for largest files: /home/nada
=== Top 15 largest files in /home/nada ===
160M  /home/nada
132M  /home/nada/snap
122M  /home/nada/snap/firefox/common
122M  /home/nada/snap/firefox
77M   /home/nada/snap/firefox/common/.cache
75M   /home/nada/snap/firefox/common/.cache/mozilla/firefox/ixku8ee2.default
75M   /home/nada/snap/firefox/common/.cache/mozilla/firefox
75M   /home/nada/snap/firefox/common/.cache/mozilla
46M   /home/nada/snap/firefox/common/.mozilla/firefox
46M   /home/nada/snap/firefox/common/.mozilla
45M   /home/nada/snap/firefox/common/.mozilla/firefox/ixku8ee2.default
45M   /home/nada/snap/firefox/common/.cache/mozilla/firefox/ixku8ee2.default/cache2/entries
45M   /home/nada/snap/firefox/common/.cache/mozilla/firefox/ixku8ee2.default/cache2
27M   /home/nada/.cache/tracker3/files
27M   /home/nada/.cache/tracker3
```

3. Show files modified within the last 24 hours.

```
===== File System Monitor Tool =====
1) Display disk usage for each mounted filesystem
2) List the top 15 largest files in a directory
3) Show files modified in the last 24 hours
4) Clean temporary files if directory size exceeds a limit
0) Exit
Choose an option: 3
Enter directory to search for files modified in last 24 hours: /home/nada
=== Files modified in last 24 hours in /home/nada ===
/home/nada/snap/snap-store/common/.cache/gnome-software/odrs/ratings.json
/home/nada/snap/snap-store/common/.cache/gnome-software/appstream/components.xmlb
/home/nada/.local/share/gvfs-metadata/home
/home/nada/.local/share/gvfs-metadata/root-as87fd4b.log
/home/nada/.local/share/gvfs-metadata/home-1a1875c5.log
/home/nada/.local/share/gvfs-metadata/root
/home/nada/.local/share/gnome-shell/application_state
/home/nada/.local/share/recently-used.xbel
/home/nada/.local/share/evolution/addressbook/system/contacts.db
/home/nada/OS_Project/file_system_monitor_tool.sh
/home/nada/OS_Project/memory_manager_tool.sh
/home/nada/.config/ibus/bus/ccf8bba185284d669187124cd112fd90-unix-wayland-0
/home/nada/.config/ibus/bus/ccf8bba185284d669187124cd112fd90-unix-0
/home/nada/.config/gedit/accels
/home/nada/.config/dconf/user
/home/nada/.config/pulse/ccf8bba185284d669187124cd112fd90-default-sink
/home/nada/.config/pulse/ccf8bba185284d669187124cd112fd90-default-source
/home/nada/.config/gtk-3.0/bookmarks
/home/nada/.cache/tracker3/files/http%3A%2F%2Ftracker.apl.gnome.org%2Fontology%2Fv3%2Ftracker%23Documents.db-shm
/home/nada/.cache/tracker3/files/http%3A%2F%2Ftracker.apl.gnome.org%2Fontology%2Fv3%2Ftracker%23Documents.db-wal
/home/nada/.cache/tracker3/files/http%3A%2F%2Ftracker.apl.gnome.org%2Fontology%2Fv3%2Ftracker%23Video.db-wal
/home/nada/.cache/tracker3/files/http%3A%2F%2Ftracker.apl.gnome.org%2Fontology%2Fv3%2Ftracker%23FileSystem.db-wal
/home/nada/.cache/tracker3/files/http%3A%2F%2Ftracker.apl.gnome.org%2Fontology%2Fv3%2Ftracker%23Pictures.db-wal
/home/nada/.cache/tracker3/files/meta.db-shm
/home/nada/.cache/tracker3/files/http%3A%2F%2Ftracker.apl.gnome.org%2Fontology%2Fv3%2Ftracker%23Pictures.db-shm
/home/nada/.cache/tracker3/files/http%3A%2F%2Ftracker.apl.gnome.org%2Fontology%2Fv3%2Ftracker%23Audio.db-shm
```

4. Clean temporary files if they exceed specified size.

```
===== File System Monitor Tool =====
1) Display disk usage for each mounted filesystem
2) List the top 15 largest files in a directory
3) Show files modified in the last 24 hours
4) Clean temporary files if directory size exceeds a limit
0) Exit
Choose an option: 4
Enter temporary directory (e.g., /tmp): /tmp
Enter max allowed size in MB: 500
Current size of /tmp: 77868 bytes
Limit: 524288000 bytes (500 MB)
Size is within limit. No clean-up needed.
```

Report for Part D

1. **df -h**: Displays disk usage for all mounted filesystems in a human-readable format. Used to give the user a clear overview of available and used space.
2. **du -ah + sort -rh | head -n 15**: Calculates file sizes recursively, sorts them from largest to smallest, and displays the top 15 files. Helps identify which files are consuming the most storage.
3. **find DIR -type f -mtime -1**: Searches for files modified within the last 24 hours. This allows users to track recent activity or changes inside a directory.
4. **du -sb**: Returns the total size of a directory in bytes. Used to compare the actual directory size with a user-defined limit.
5. **find DIR -type f -name '*.tmp' -delete**: Locates temporary files (e.g., *.tmp, *.temp) and removes them when the directory exceeds the specified size. This helps automate cleanup of unnecessary files.

Learning Outcomes

Through this project, we gained a clearer understanding of how Linux handles disk usage, file sizes, and filesystem organization. Working with commands like *df*, *du*, and *find* improved our ability to monitor storage and identify which files or directories contribute most to disk consumption. Implementing cleanup logic helped demonstrate how administrators can manage temporary files to maintain system efficiency.

Part E — *network_monitor_tool.sh*

This shell script was developed to help monitor, analyze, and manage network activity on a Linux system. It provides several essential capabilities, including viewing all active network interfaces with their corresponding IP addresses, measuring bandwidth usage per interface, monitoring network connections for a specific IP, and tracking network traffic over time with detailed logging. These features make the tool useful for both troubleshooting and general network administration.

1. Display the IP addresses and status of active network interfaces.

```
===== Network Monitor Tool =====
1) Display IP addresses and status of active network interfaces
2) Show bandwidth usage for each interface (sampled)
3) Monitor network connections and alert if a specific IP connects
4) Track network traffic over time and log data
0) Exit
Choose an option: 1
=== Active network interfaces and IP addresses ===
lo                UNKNOWN    127.0.0.1/8 ::1/128
ens33             UP          192.168.146.130/24 fe80::95b:a0da:4a46:66bd/64
```

2. Show bandwidth usage for each interface.

```
===== Network Monitor Tool =====
1) Display IP addresses and status of active network interfaces
2) Show bandwidth usage for each interface (sampled)
3) Monitor network connections and alert if a specific IP connects
4) Track network traffic over time and log data
0) Exit
Choose an option: 2
Enter sampling interval in seconds (default: 1):
Measuring RX/TX for each interface over 1s...

Interface: ens33
RX: 0 KB/s
TX: 0 KB/s
```

3. Monitor network connections and alert if a specific IP connects.

```

===== Network Monitor Tool =====
1) Display IP addresses and status of active network interfaces
2) Show bandwidth usage for each interface (sampled)
3) Monitor network connections and alert if a specific IP connects
4) Track network traffic over time and log data
0) Exit
Choose an option: 3
Enter IP address to monitor for connections: 8.8.8.8
Polling interval in seconds (default: 5):
Monitoring active connections for IP: 8.8.8.8
Press Ctrl + C to stop.
[Wed 26 Nov 2025 02:31:54 PM EST] No active TCP connection with 8.8.8.8.
[Wed 26 Nov 2025 02:31:59 PM EST] No active TCP connection with 8.8.8.8.
[Wed 26 Nov 2025 02:32:05 PM EST] No active TCP connection with 8.8.8.8.

```

4. Track network traffic over time and log data.

```

===== Network Monitor Tool =====
1) Display IP addresses and status of active network interfaces
2) Show bandwidth usage for each interface (sampled)
3) Monitor network connections and alert if a specific IP connects
4) Track network traffic over time and log data
0) Exit
Choose an option: 4
Enter interface to monitor (e.g., eth0, ens33, wlan0): ens33
Enter total duration in seconds: 30
Enter sampling interval in seconds (default: 1): 1
Enter log file path (default: network_traffic.log):
Tracking traffic on interface ens33 for 30 seconds...
Logging to: network_traffic.log
[2025-11-26 14:35:01] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:02] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:03] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:04] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:05] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:06] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:07] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:08] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:09] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:10] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:11] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:12] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:13] ens33 RX: 775 B/s, TX: 495 B/s
[2025-11-26 14:35:15] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:16] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:17] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:18] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:19] ens33 RX: 60 B/s, TX: 60 B/s
[2025-11-26 14:35:20] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:21] ens33 RX: 0 B/s, TX: 0 B/s
[2025-11-26 14:35:22] ens33 RX: 0 B/s, TX: 0 B/s

```

Report for Part E

1. `ip -br addr show up / ifconfig`: Used to display active network interfaces along with their assigned IP addresses. The `ip` command is preferred, while `ifconfig` serves as a fallback for older systems.
2. `/sys/class/net/*/statistics/rx_bytes & tx_bytes`: Reads raw byte counters directly from the kernel for each network interface. These are used to calculate real-time bandwidth usage (RX/TX rates).
3. `ss -nt` or `netstat -nt`: Displays active TCP network connections. The script checks these outputs to detect whether a specific IP address has connected, triggering an alert when detected.
4. `find /sys/class/net/$IFACE/statistics`: Accesses network interface statistics to compute traffic rates when tracking data over time.
5. `network_traffic.log`: Appends timestamped traffic data (RX, TX, and rates) to a log file. This allows long-term monitoring, review, or graphing of network activity.

Learning Outcomes

This part of the project strengthened our understanding of how Linux handles networking at a low level, particularly by exposing us to kernel-level counters for traffic measurement and the structure of network interfaces. We learned how to read and interpret IP assignments, bandwidth statistics, and TCP connections, which are all essential for diagnosing network issues. Building alerting features and long-term logging also helped show how administrators can track unusual activity or performance drops in real time.